In [164…

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.metrics import r2_score, mean_squared_error
```

In [7]:

```python
df = pd.read_csv('insurance.csv')
df.head()
```

Out[7]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

In [5]:

```python
#EDA
```

In [7]:

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1338 non-null   int64
 1   sex       1338 non-null   object
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64
 4   smoker    1338 non-null   object
 5   region    1338 non-null   object
 6   charges   1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

In [9]:

```python
df.describe()
```

Out[9]:

	age	bmi	children	charges
count	1338.000000	1338.000000	1338.000000	1338.000000
mean	39.207025	30.663397	1.094918	13270.422265
std	14.049960	6.098187	1.205493	12110.011237
min	18.000000	15.960000	0.000000	1121.873900
25%	27.000000	26.296250	0.000000	4740.287150
50%	39.000000	30.400000	1.000000	9382.033000
75%	51.000000	34.693750	2.000000	16639.912515
max	64.000000	53.130000	5.000000	63770.428010

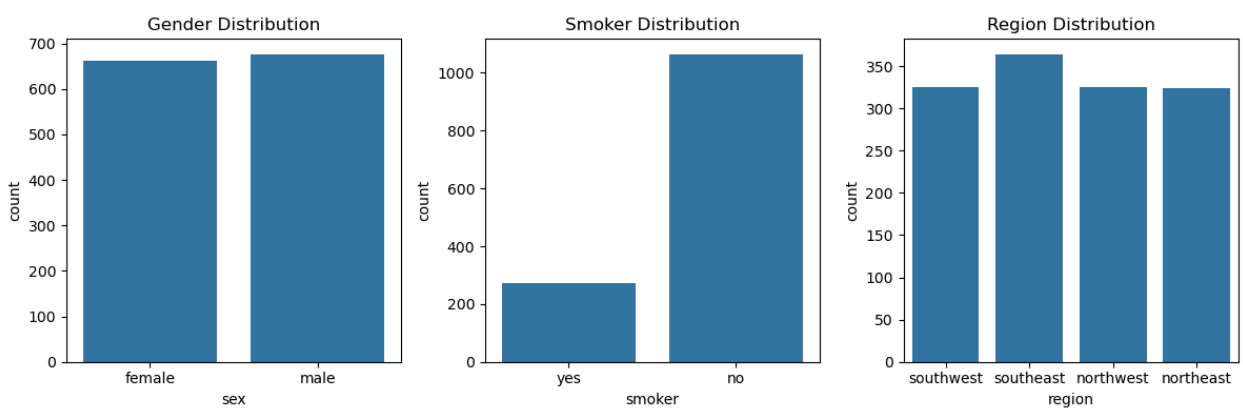
In [13]:

```
plt.figure(figsize = (12,4))
plt.subplot(1, 3, 1)
sns.countplot(data = df, x = 'sex')
plt.title('Gender Distribution')

plt.subplot(1, 3, 2)
sns.countplot(data = df, x = 'smoker')
plt.title('Smoker Distribution')

plt.subplot(1,3,3)
sns.countplot(data = df, x = 'region')
plt.title('Region Distribution')

plt.tight_layout()
plt.show()
```

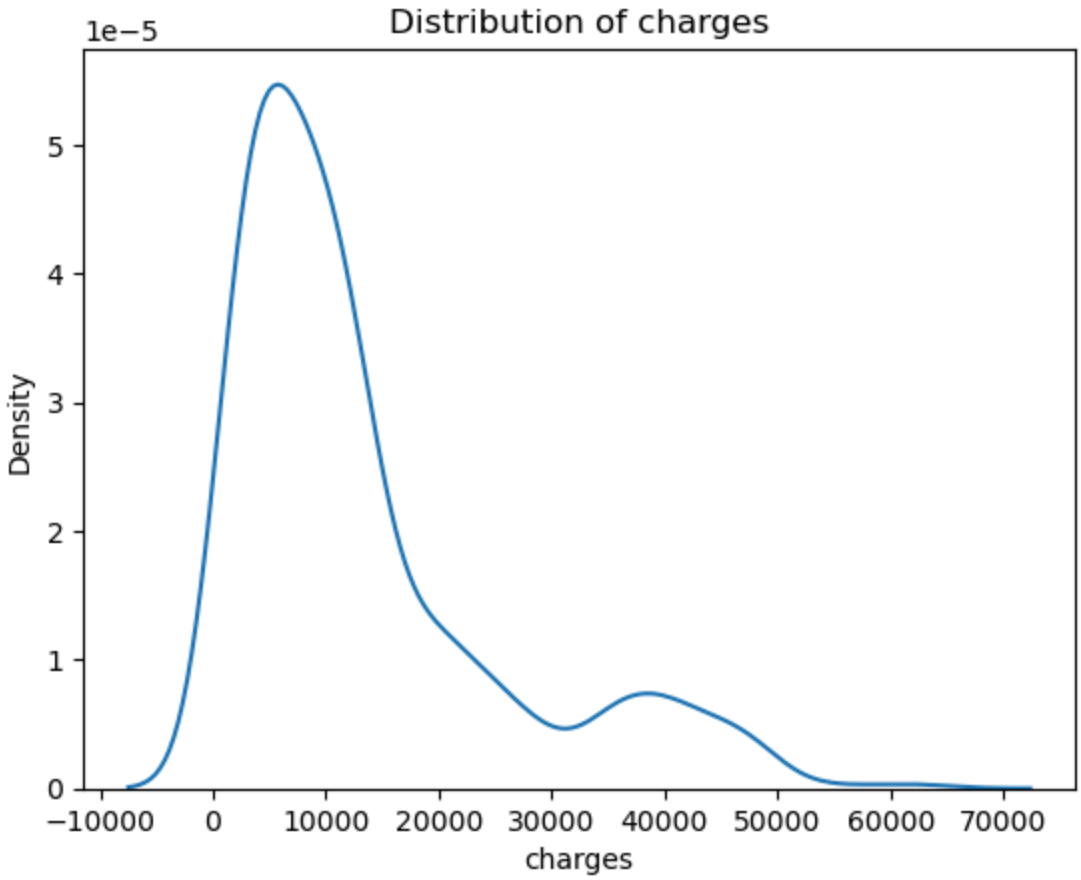


In [15]:

```
#1. Equal Distribution of male and female
#2. Non smokers are more in number than smokers
#3. Almost equal distribution across region
```

In [19]:

```
sns.kdeplot(data = df, x = 'charges')
plt.title('Distribution of charges')
plt.show()
```



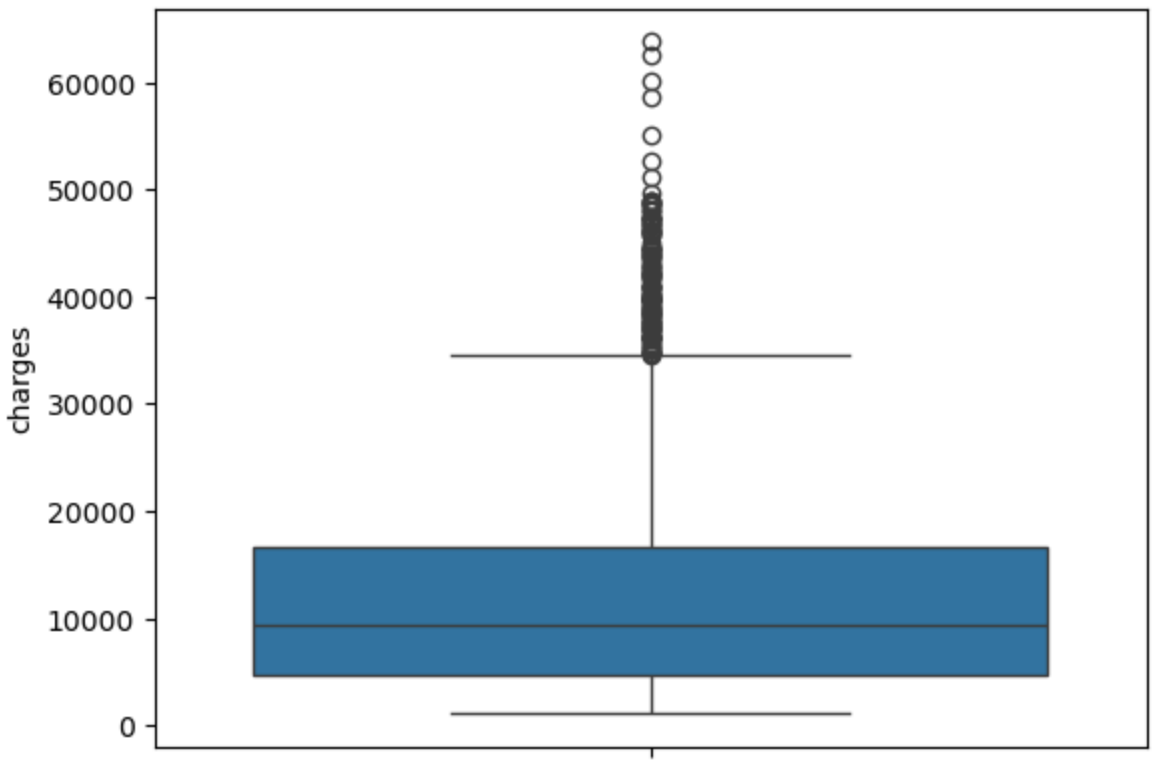
In [21]:
```
df['charges'].skew()
```

Out[21]: 1.5158796580240388

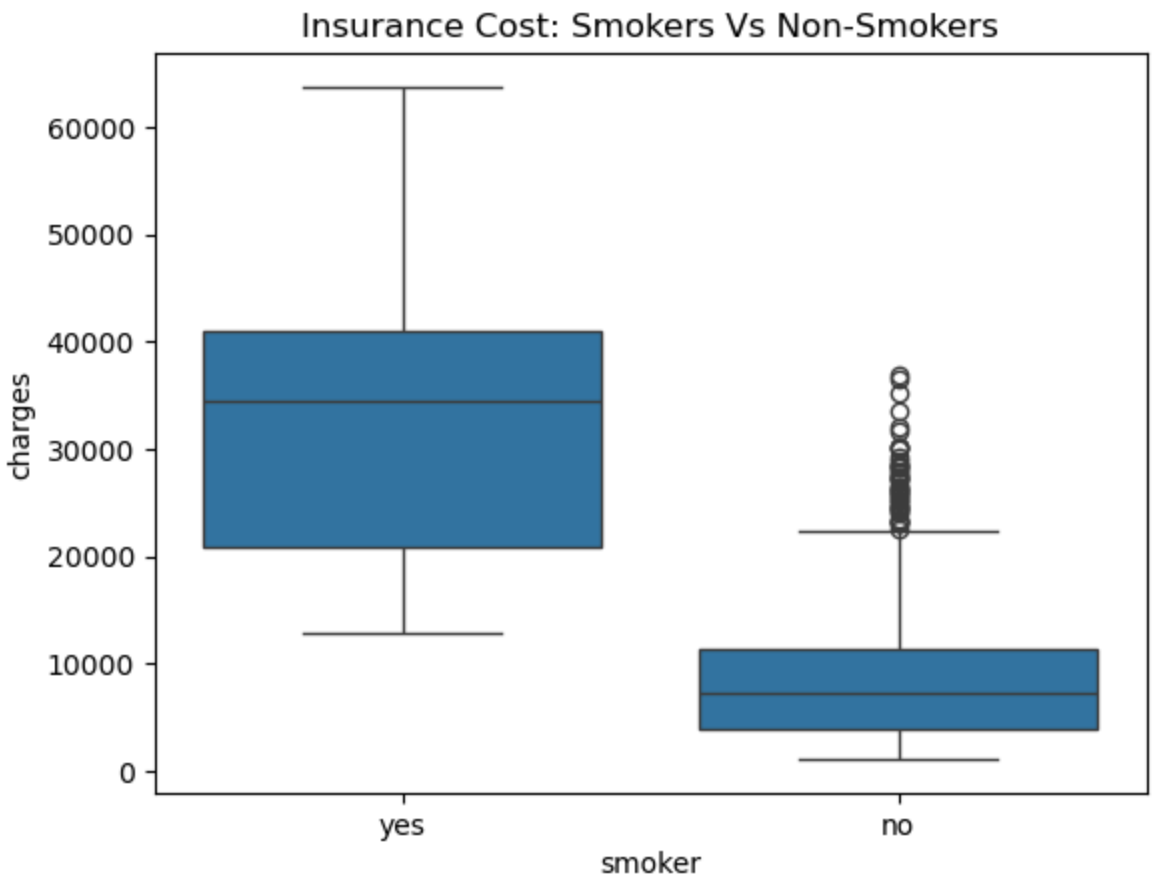
In [23]:
```
#Charges distribution is right skewed with a skew factor of 1.5
```

In [27]:
```
sns.boxplot(df['charges'])
```

Out[27]: <Axes: ylabel='charges'>



In [33]:
```
sns.boxplot(x = 'smoker', y = 'charges', data = df)
plt.title('Insurance Cost: Smokers Vs Non-Smokers')
plt.show()
```



In [35]:
```
#Median cost of insurance for smokers is 7 times more than that of No-Smokers
```

In [ ]:
```

```

In [38]:
```
#Numerical Columns
```

In [40]:
```
df.head(2)
```

Out[40]:

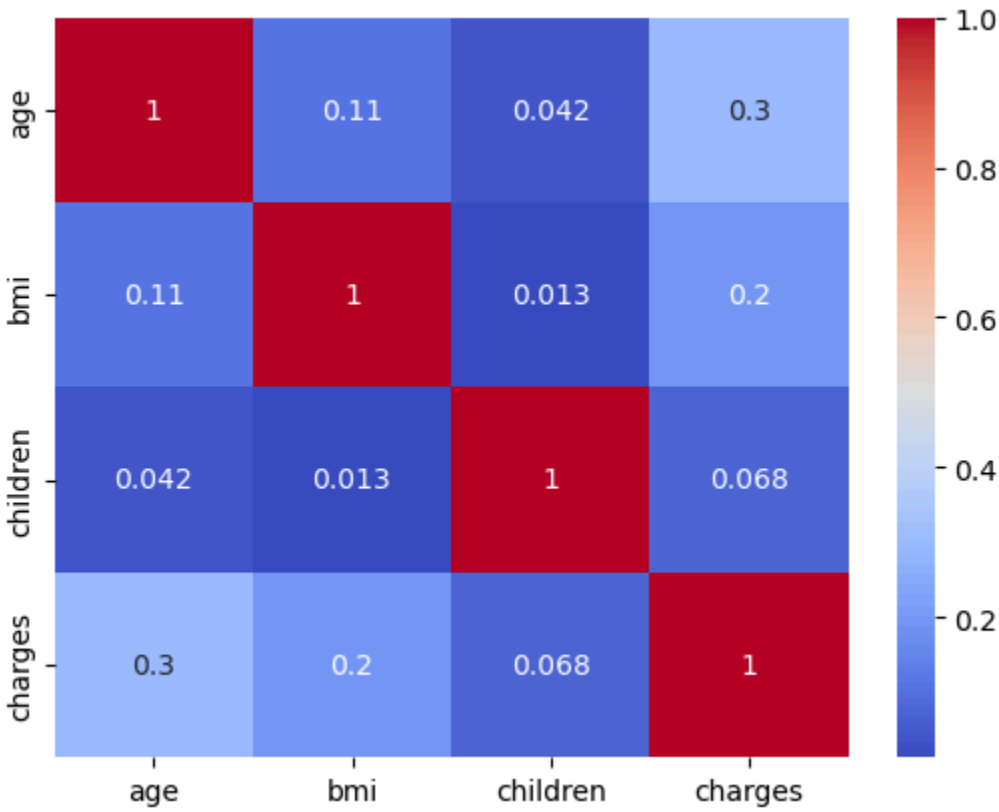
	age	sex	bmi	children	smoker	region	charges
0	19	female	27.90	0	yes	southwest	16884.9240
1	18	male	33.77	1	no	southeast	1725.5523

In [42]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1338 non-null   int64
 1   sex       1338 non-null   object
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64
 4   smoker    1338 non-null   object
 5   region    1338 non-null   object
 6   charges   1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

In [60]:
```
numeric_data = df.select_dtypes(include = ['int64', 'float64'])
sns.heatmap(numeric_data.corr(), annot = True, cmap ='coolwarm')
```

Out[60]: <Axes: >

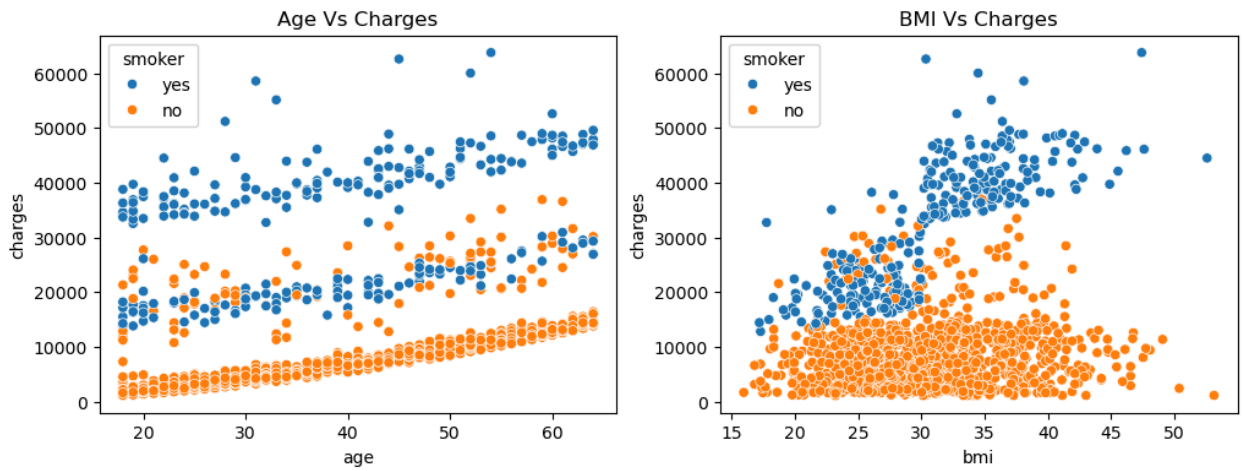


In [62]:
```
#BMI and Age has a moderate correlation with charges
#Children has almost no correlation with charges
```

In [68]:
```
plt.figure(figsize = (12,4))
plt.subplot(1,2,1)
sns.scatterplot(x = 'age', y = 'charges', data = df, hue = 'smoker')
plt.title('Age Vs Charges')
```

```
plt.subplot(1,2,2)
sns.scatterplot(x='bmi', y='charges', data = df, hue = 'smoker')
plt.title('BMI Vs Charges')
```

Out[68]: Text(0.5, 1.0, 'BMI Vs Charges')



In [ ]:

In [ ]: *#Data Preprocessing*

In [72]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       1338 non-null   int64
 1   sex       1338 non-null   object
 2   bmi       1338 non-null   float64
 3   children  1338 non-null   int64
 4   smoker    1338 non-null   object
 5   region    1338 non-null   object
 6   charges   1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

In [78]:
```
df_encoded = pd.get_dummies(df, columns = ['sex', 'smoker', 'region'], drop_fi
df_encoded
```

Out[78]:

	age	bmi	children	charges	sex_male	smoker_yes	region_northwe
0	19	27.900	0	16884.92400	False	True	Fa
1	18	33.770	1	1725.55230	True	False	Fa
2	28	33.000	3	4449.46200	True	False	Fa
3	33	22.705	0	21984.47061	True	False	Tr
4	32	28.880	0	3866.85520	True	False	Tr
...	...	...	...	...	...	...	
1333	50	30.970	3	10600.54830	True	False	Tr
1334	18	31.920	0	2205.98080	False	False	Fa
1335	18	36.850	0	1629.83350	False	False	Fa
1336	21	25.800	0	2007.94500	False	False	Fa
1337	61	29.070	0	29141.36030	False	True	Tr

1338 rows × 9 columns

In [80]:
```
df_encoded.columns
```

Out[80]:
```
Index(['age', 'bmi', 'children', 'charges', 'sex_male', 'smoker_yes',
       'region_northwest', 'region_southeast', 'region_southwest'],
      dtype='object')
```

In [ ]:

In [83]:
```
#Model Building
```

In [85]:
```
X = df_encoded.drop('charges', axis = 1)
X
```

Out[85]:

	age	bmi	children	sex_male	smoker_yes	region_northwest	region_sc
0	19	27.900	0	False	True	False	
1	18	33.770	1	True	False	False	
2	28	33.000	3	True	False	False	
3	33	22.705	0	True	False	True	
4	32	28.880	0	True	False	True	
...	...	...	...	...	...	...	
1333	50	30.970	3	True	False	True	
1334	18	31.920	0	False	False	False	
1335	18	36.850	0	False	False	False	
1336	21	25.800	0	False	False	False	
1337	61	29.070	0	False	True	True	

1338 rows × 8 columns

In [87]:
```
y = df_encoded['charges']
y
```

Out[87]:
```
0       16884.92400
1        1725.55230
2        4449.46200
3       21984.47061
4        3866.85520
           ...
1333    10600.54830
1334     2205.98080
1335     1629.83350
1336     2007.94500
1337    29141.36030
Name: charges, Length: 1338, dtype: float64
```

In [89]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, ran
```

In [91]:
```
X_train
```

Out[91]:

	age	bmi	children	sex_male	smoker_yes	region_northwest	region_sc
560	46	19.950	2	False	False	True	
1285	47	24.320	0	False	False	False	
1142	52	24.860	0	False	False	False	
969	39	34.320	5	False	False	False	
486	54	21.470	3	False	False	True	
...	...	...	...	...	...	...	
1095	18	31.350	4	False	False	False	
1130	39	23.870	5	False	False	False	
1294	58	25.175	0	True	False	False	
860	37	47.600	2	False	True	False	
1126	55	29.900	0	True	False	False	

1070 rows × 8 columns

In [101…

```
y_test
```

Out[101…

```
764      9095.06825
887      5272.17580
890     29330.98315
1293     9301.89355
259     33750.29180
           ...
109     47055.53210
575     12222.89830
535      6067.12675
543     63770.42801
846      9872.70100
Name: charges, Length: 268, dtype: float64
```

In [ ]:

In [104…

```
#Feature Scaling
```

In [215…

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
```

In [231…

```
X_train_scaled = sc.fit_transform(X_train)
```

In [112…

```
X_train_scaled
```

Out[112…
```
array([[ 0.47222651, -1.75652513,  0.73433626, ...,  1.78316783,
        -0.59966106, -0.5723141 ],
       [ 0.54331294, -1.03308239, -0.91119211, ..., -0.56079971,
        -0.59966106, -0.5723141 ],
       [ 0.8987451 , -0.94368672, -0.91119211, ..., -0.56079971,
         1.66760869, -0.5723141 ],
       ...,
       [ 1.3252637 , -0.89153925, -0.91119211, ..., -0.56079971,
        -0.59966106, -0.5723141 ],
       [-0.16755139,  2.82086429,  0.73433626, ..., -0.56079971,
        -0.59966106,  1.74729228],
       [ 1.1120044 , -0.10932713, -0.91119211, ..., -0.56079971,
        -0.59966106,  1.74729228]])
```

In [114…
```
X_test_scaled = sc.transform(X_test)
```

In [116…
```
X_test_scaled
```

Out[116…
```
array([[ 0.40114007, -0.89153925,  0.73433626, ..., -0.56079971,
        -0.59966106, -0.5723141 ],
       [-0.23863782, -0.08946143, -0.91119211, ...,  1.78316783,
        -0.59966106, -0.5723141 ],
       [ 1.75178229, -0.60845296, -0.91119211, ...,  1.78316783,
        -0.59966106, -0.5723141 ],
       ...,
       [-0.09646495, -0.41972876, -0.08842793, ..., -0.56079971,
        -0.59966106, -0.5723141 ],
       [ 1.04091797,  2.78941026, -0.91119211, ..., -0.56079971,
         1.66760869, -0.5723141 ],
       [ 0.82765867,  0.60252728, -0.08842793, ..., -0.56079971,
        -0.59966106,  1.74729228]])
```

In [ ]:

In [119…
```
#Model Training
```

In [121…
```
lr = LinearRegression()
```

In [123…
```
lr.fit(X_train_scaled, y_train)
```

Out[123…
```
▾ LinearRegression
LinearRegression()
```

In [125…
```
y_pred = lr.predict(X_test_scaled)
```

In [127…
```
y_pred
```

Out[127…

```
array([ 8969.55027444,  7068.74744287, 36858.41091155,  9454.67850053,
       26973.17345656, 10864.11316424,   170.28084136, 16903.45028662,
        1092.43093614, 11218.34318352, 28101.68455267,  9377.73460205,
        5263.0595179 , 38416.04221107, 40255.82339284, 37098.25353123,
       15240.39392306, 35912.88264434,  9112.52398703, 31461.92108909,
        3847.68845883, 10130.12001517,  2370.54189389,  7140.21550828,
       11301.76782638, 12961.65366224, 14509.47251876,  6159.8976107 ,
        9963.85857263,  2177.85718217,  9115.93673493, 13073.68932159,
        4561.82376202,  3408.20756033,  4459.81359745, 13032.06505076,
        1979.99357292,  8813.28303302, 33271.29124448, 32585.51583927,
        3908.76090964,  4326.10774721, 14142.81326533, 11423.45494846,
        8774.13955311, 12097.28051001,  5281.57353499,  3150.5596042 ,
       35494.46461214,  9150.1124786 , 15836.84575621,  2343.57470069,
       12364.78414194,  1482.29488266, 13389.06105161, 12573.57395972,
        4341.83680558, 32165.33688042, 13321.3360032 , 12896.82071102,
       14167.99421483, 10506.17623512, 16360.78543548,  7763.89824584,
       11839.25019431,  4061.19750503, 26652.40230125, 10930.14138671,
        2137.41385988,  6209.01123411, 10729.82391284, 11628.3104129 ,
       10981.04528946,  9166.50818596, 11954.27732874,  6747.85121734,
        7248.5304713 , 10735.16710748,  6580.84819774,  8762.00329355,
        3767.13383454, 36632.4975496 ,  6378.11979721, 30842.09248656,
       34846.52451051, 35278.07387112,  7019.444352  , 12861.38414264,
        9942.30149778, 14473.5260648 , 17693.37304474, 35258.24845137,
       33029.58968269,  6185.91730447, 31999.98962535,  9481.33158273,
       29444.04271523,  3674.48498404, 28308.26432106,  5823.36495229,
        5407.76752001,  1883.4947576 , 11499.675042  , 15075.90690632,
       11699.63163008,  4308.82427855,  9895.1840044 , 31708.40056201,
         -86.87094667, 32819.71429004,  3280.69178415, 10183.88853878,
       14318.76389179, 31642.35684542, 11461.57806791,  3929.23701831,
       13107.89313088, 31810.99450607,  8152.02593593,  3238.08417076,
        8439.56108376, 10594.63871458, 15219.68736374,  5647.8808143 ,
        3781.95285499, 10228.944897  , 10900.12933883, 11122.74845192,
       14438.14112575,  7430.31504776,  5386.22676759,  9231.32739901,
        9343.76283713, 12538.27606344,  8337.66982683, 15333.36900871,
        8411.2145439 , 31797.27496298, 35785.91843418, 31603.71967017,
        6011.96229251, 12607.03584641,  6013.5115031 , 14560.79590559,
        2493.47989441, 32963.45524228,  6265.14380504,  5034.62173797,
       14344.81347407,  6941.1412259 , 38670.01270366,  3087.58741836,
        5885.8752536 , 31686.24200595, 11562.61859836,  8476.04749512,
       14806.72486264,  9814.46186143, 27105.71831469, 33453.83352069,
       14551.8999207 ,  1684.36856768, 13166.96197398,  2222.76894041,
        5449.59393727, 11568.96325488, 39807.96912709, 36500.65163031,
       34001.37945748,  3897.27856532,  7456.14132125,  8661.82084477,
       12450.92458882,  4813.53293089,  2047.65528159, 32112.11251984,
       25111.52085938, 17484.27663755, 26411.46181822, 10159.52421   ,
       37260.32666386,  -441.23918333,  6779.55013103,  7781.45337795,
        4367.95988484,  5105.87170813,  5919.18675042,  4305.71645941,
       15191.08806502, 11132.09935114,  6932.80116584,  2525.64793222,
        1536.05183213, 31944.78284317, 16414.12251517, 12011.53367195,
        1268.05926603, 12531.25953189,  1564.93415917,  8737.33621694,
        1873.03940488, 33916.22971211, 10858.38635063,  2603.43633853,
       25674.40250332, 26343.43022704,  9430.91152033,  1800.73500777,
       13261.42480211,  1120.17810533, 10386.66427709, 10567.29006474,
       16944.25995713, 26846.54662457,  6939.11178393,  5193.04710054,
```

```
 5846.00017265, 13229.60536846, 11098.33930228,  8362.28134289,
 5135.53940151, 12308.34064139, 13861.17886997, 35773.70926219,
 4157.01930317, 28917.86562624,  -914.37342357,  2873.71150671,
11046.2540774 , 15683.06950225,  5210.67532324,  6888.38518351,
 3854.31140958, 31312.64705453,  7241.43226665, 12405.99508651,
 5619.17039188,  9528.22557021, 36314.009043  ,  4429.40596906,
 9667.91523953, 31161.15738995,  5747.13292318,  4603.37294255,
 1048.35533791,  4832.6604097 ,  4574.9041044 ,  6507.30666036,
18659.12407756, -1545.57184934,  2376.4352498 , 10694.62157146,
 3151.28919904, 10209.96361187,  3733.89128353,  5125.08103172,
12400.90700504,  6218.65296628,  8231.63765089,  7590.50155269,
 8924.15352268, 10482.90359975, 27808.04576398, 39061.50093248,
11761.4991981 ,  7687.56363151, 40920.29151165, 12318.58665305])
```

In [129…

```
y_test
```

Out[129…

```
764      9095.06825
887      5272.17580
890     29330.98315
1293     9301.89355
259     33750.29180
           ...
109     47055.53210
575     12222.89830
535      6067.12675
543     63770.42801
846      9872.70100
Name: charges, Length: 268, dtype: float64
```

In [ ]:

In [134…

```
#Model Evaluation
```

In [144…

```
print('r2_score:', r2_score(y_test, y_pred))
print('MSE:', mean_squared_error(y_test, y_pred))
```

```
r2_score: 0.7835929767120723
MSE: 33596915.85136147
```

In [146…

```
#Improving Model
```

In [150…

```
ridge = Ridge(alpha = 1.0)
```

In [152…

```
ridge.fit(X_train_scaled, y_train)
```

Out[152…

```
▾ Ridge
Ridge()
```

In [154…

```
ridge.predict(X_test_scaled)
```

Out[154…

```
array([ 8971.92398524,  7073.34230054, 36833.99117804,  9459.05708554,
       26960.13603638, 10867.25311901,   181.56953724, 16899.75722576,
        1101.82745503, 11221.03994558, 28088.58085219,  9382.00947429,
        5271.94742923, 38394.98039692, 40231.91711466, 37078.09337945,
       15239.98494731, 35892.18713736,  9117.51182438, 31442.90668116,
        3858.59038978, 10134.01900387,  2381.50951787,  7144.20211055,
       11303.02859733, 12959.60430429, 14507.08052805,  6166.37426771,
        9965.27734524,  2190.86702555,  9120.06094624, 13072.32133633,
        4571.24578983,  3417.13975999,  4468.99719689, 13031.4861873 ,
        1990.22144783,  8816.76712733, 33250.70541708, 32569.44803572,
        3918.40085934,  4335.14420688, 14141.12884982, 11424.89280529,
        8781.3440694 , 12098.74221108,  5288.702707  ,  3160.19493063,
       35476.1456027 ,  9157.01636076, 15835.01544321,  2356.71657806,
       12364.72245268,  1492.89111816, 13390.96476072, 12573.87330066,
        4349.54968474, 32148.13038759, 13320.07655316, 12894.85269141,
       14165.84931854, 10510.52188754, 16358.86443049,  7768.29230695,
       11838.27314662,  4069.00748525, 26641.10783394, 10935.23754218,
        2147.05237812,  6216.39591644, 10731.79197321, 11628.22600087,
       10983.99327872,  9173.22158343, 11958.04278425,  6754.941224  ,
        7254.49741675, 10735.96381487,  6588.83444671,  8766.05826743,
        3778.64357735, 36608.81647291,  6384.36034102, 30823.80260417,
       34828.69202935, 35255.89702892,  7028.60635157, 12861.59217288,
        9944.67758133, 14473.50979453, 17687.38036529, 35237.49604321,
       33008.48515928,  6192.48629995, 31984.52413053,  9485.86891389,
       29430.94113671,  3684.74200246, 28293.27633299,  5828.84080199,
        5414.29978883,  1893.57103964, 11503.63896325, 15075.27288362,
       11699.92273645,  4320.16542122,  9901.12006149, 31692.47971405,
         -74.83690326, 32801.72142013,  3290.82544866, 10189.67096876,
       14315.66717816, 31622.80855183, 11461.2967537 ,  3939.03837486,
       13106.91579674, 31792.58538788,  8158.37513932,  3248.09564075,
        8442.37779792, 10597.79855213, 15216.18061979,  5657.57952037,
        3790.3025827 , 10231.102198  , 10903.54501281, 11124.11522066,
       14437.68015984,  7438.40246332,  5394.65214521,  9238.35301314,
        9349.06393161, 12536.8120355 ,  8341.39357161, 15331.86552057,
        8415.1329028 , 31779.67020802, 35762.92006647, 31583.85710233,
        6017.52546979, 12605.37721858,  6022.83132077, 14558.74272371,
        2506.04700766, 32945.19615938,  6272.44616854,  5041.54748775,
       14343.26400596,  6947.8538203 , 38645.02591755,  3096.11529033,
        5895.48748434, 31668.03498371, 11563.33578918,  8479.88451567,
       14804.14704391,  9818.78458881, 27093.42294342, 33433.83141396,
       14548.87346629,  1694.55380204, 13167.61775425,  2232.57702495,
        5458.50416881, 11573.40951342, 39781.22606491, 36476.71323617,
       33980.78791021,  3905.37804519,  7460.73735217,  8665.53145282,
       12451.00981355,  4821.9253745 ,  2057.51534723, 32096.23723052,
       25100.54663283, 17482.25248805, 26397.88410328, 10160.82643941,
       37234.71076474,  -429.99997306,  6785.79021357,  7786.08031963,
        4376.52239732,  5112.52831763,  5925.76013433,  4313.33202908,
       15190.48425404, 11133.89190437,  6940.35235232,  2533.95053837,
        1545.17346262, 31927.85194599, 16413.12280049, 12013.47163896,
        1278.62595962, 12530.17229687,  1574.11705587,  8740.45249262,
        1882.61435462, 33896.10667375, 10863.74689454,  2614.40093846,
       25663.68057592, 26331.65786975,  9434.37251508,  1811.31557682,
       13263.62092429,  1131.10608813, 10389.34371835, 10570.70365775,
       16939.30460406, 26835.67281784,  6947.84418542,  5198.64240845,
```

```
 5854.26621168, 13231.10340632, 11100.91662419,  8368.19787817,
 5142.23462492, 12309.29667834, 13863.91834114, 35754.85050525,
 4165.92681759, 28905.05002928,  -901.73348835,  2883.8606988 ,
11050.69801496, 15682.82266609,  5220.84043353,  6892.81631465,
 3862.93028693, 31295.46442563,  7245.51797228, 12408.95309603,
 5627.21818192,  9532.13024236, 36293.33953109,  4440.26501445,
 9668.75229861, 31144.98838478,  5753.18002001,  4612.93111487,
 1060.17708204,  4840.8611115 ,  4582.88464396,  6514.48257025,
18656.36839906, -1531.37332072,  2386.03792366, 10698.35299117,
 3160.00642199, 10210.36979657,  3742.19856471,  5134.18310184,
12401.78468513,  6225.86757863,  8235.36963095,  7594.72980595,
 8927.81220008, 10483.82178974, 27796.26901439, 39038.62319963,
11761.31162429,  7692.31927862, 40894.57036796, 12318.68341803])
```

In [158…

```
y_pred_ridge = ridge.predict(X_test_scaled)
print('R2_score of Ridge model:', r2_score(y_test, y_pred_ridge))
```

```
R2_score of Ridge model: 0.7835410749121385
```

In [168…

```
lasso = Lasso(alpha = 1.0)
lasso.fit(X_train_scaled, y_train)
y_pred_lasso = lasso.predict(X_test_scaled)
print('R2_score of Lasso model:', r2_score(y_test, y_pred_lasso))
```

```
R2_score of Lasso model: 0.7835657290476568
```

In [ ]:

```

```

In [171…

```
#Real World Data
```

In [173…

```
df.head(3)
```

Out[173…

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.90	0	yes	southwest	16884.9240
1	18	male	33.77	1	no	southeast	1725.5523
2	28	male	33.00	3	no	southeast	4449.4620

In [235…

```
#Predicting the insurance premium for a new client
new_client = [30, 26, 0, True, True, False, False, True]
```

In [237…

```
new_df = pd.DataFrame([new_client], columns = ['age', 'bmi', 'children', 'sex_
new_df
```

Out[237…

	age	bmi	children	sex_male	smoker_yes	region_northwest	region_southea
0	30	26	0	True	True	False	Fal

In [239…

```
sc.transform(new_df)
```

Out[239…
```
array([[-0.66515641, -0.75496253, -0.91119211,  0.97598911,  1.96561348,
        -0.56079971, -0.59966106,  1.74729228]])
```

In [241…
```
lr.predict(sc.transform(new_df))
```

Out[241…
```
array([27365.19628522])
```

In [243…
```
print('The insurance premium of the concerned:', lr.predict(sc.transform(new_d
```

```
The insurance premium of the concerned: 27365.196285215166
```

In [ ]: